PENSE-BÊTE · TECHNIQUE ET COMMANDES

Gestion des processus et de l'ordonnancement

06-170 PSE 25-27 2-1 Les Fonctions du Noyau La Gestion des Processus — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. Un processus est un programme dynamique en cours d'exécution avec ses ressources et un cycle de vie.
2. L'ordonnanceur décide de l'utilisation du processeur et gère la commutation de contexte via le PCB.
3. Les threads partagent la mémoire du processus parent contrairement aux processus lourds qui sont isolés.

Définition et structure d'un processus

Notion	À retenir
Concept fondamental	Programme en cours d'exécution avec ses ressources. Cycle de vie : **création**, **exécution**, **attente** et **fin**. Le **système d'exploitation** garantit l'isolation, la sécurité et la répartition du processeur.
Composition interne	Contient : **code**, **espace mémoire** (**pile**/stack : appels et variables locales ; **tas**/heap : allocations dynamiques ; zone de données), **descripteurs** (fichiers, dossiers, réseau), **contexte d'exécution** (registres, pointeurs) et **état** (prêt, élu, bloqué).

Processus lourds vs Threads

Notion	À retenir
Processus lourds	Environnement d'exécution propre, consommation mémoire élevée, espace mémoire **totalement isolé**, communication via **IPC**.
Threads	Tâches parallèles au sein d'un même processus, **partagent la mémoire du parent**, contexte d'exécution propre, améliorent réactivité, performance et exploitation **multicoeurs**.

Le rôle de l'ordonnanceur

Notion	À retenir
Fonctions principales	Décide de l'utilisation du processeur et de la durée via un **quantum** ; interrompt si **quantum terminé** ou **bloqué** ; gère le contexte via le **PCB**.
Architecture à deux niveaux	Haut niveau : gère la **stratégie globale** et les **priorités** ; Bas niveau (**dispatcher**) : assure la **commutation de contexte** très rapide.

Modes de multiprogrammation

Notion	À retenir
Multiprogrammation non préemptive	Le processus garde le processeur jusqu'à son **blocage** ou sa **fin volontaire** ; simple mais risque de **monopolisation** ; peu adapté à l'**interactif**.
Multiprogrammation préemptive	Le noyau peut **interrompre** un processus à tout moment ; assure **équité**, **réactivité** et **isolation** ; utilisée par les **systèmes modernes** pour le multitâche.

Gestion du contexte et PCB

Notion	À retenir
Contexte matériel	Regroupe les infos liées au processeur : **registres** (valeurs intermédiaires), **compteur ordinal** (prochaine instruction), **pointeur de pile** et **mode d'exécution** (utilisateur ou noyau).
Contexte logiciel	Informations gérées par l'**OS** : **priorité** (ordre de passage), **liste des fichiers ouverts** et **droits d'accès**.
Process Control Block	Structure mémoire où le noyau enregistre le contexte à chaque **interruption** pour permettre la reprise de l'exécution du processus.

Algorithmes d'ordonnancement

Notion	À retenir
Algorithmes classiques	**FIFO** : ordre d'arrivée (mauvais pour l'interactif) ; **SJF** et **SRTF** : processus courts pour réduire l'attente moyenne ; **HRN** : équité via temps d'attente.
Algorithmes avancés	**Round Robin** : quantum fixe pour réactivité ; **MLF** : priorité selon comportement ; **HPF** : niveau de priorité défini ; **PDS** : priorité selon charge globale.

Communication et synchronisation

Notion	À retenir
Problématique ressources	Assurer la cohérence lors de l'accès à des ressources communes via les **sections critiques** (parties du code manipulant ces ressources).
Mécanismes synchronisation	Accès exclusif : **mutex** ; nombre d'accès simultanés : **sémaphores** ; fichiers/bases de données : **verrous** ou **locks** ; communication asynchrone : **signaux**.
Risques et échanges	**Interblocage** (**deadlock**) : attente circulaire fatale ; **famine** : processus jamais servi ; **IPC** : fichiers, mémoire partagée, pipes, sockets et signaux.

Pièges et confusions

- ⚠ Confondre le programme statique sur disque avec le processus dynamique en mémoire
- ⚠ Confondre la pile pour les variables locales avec le tas pour les allocations dynamiques
- ⚠ Oublier que le PCB est indispensable pour la sauvegarde et la restauration du contexte
- ⚠ Ignorer que les threads partagent la mémoire du processus parent contrairement aux processus lourds

Sigles et définitions

Programme	Fichier inerte contenant du code stocké sur le disque
Processus	Programme dynamique en cours d'exécution avec ses ressources et un cycle de vie
Thread	Unité d'exécution au sein d'un processus partageant sa mémoire
Quantum	Durée de temps processeur allouée à un processus par l'ordonnanceur
PCB	Structure mémoire sauvegardant le contexte vital d'un processus
Mutex	Verrou logiciel assurant l'exclusion mutuelle pour l'accès à une ressource unique
Interblocage	Situation d'attente circulaire où des processus se bloquent mutuellement
IPC	Mécanismes permettant l'échange de données entre plusieurs processus

Auto-test

1. Quelle est la différence entre un programme et un processus ?
2. À quoi sert le PCB ?
3. Qu'est-ce qu'un quantum ?
4. Quelle est la différence de gestion mémoire entre un thread et un processus lourd ?
5. Qu'est-ce qu'un interblocage ?

Réponses : **1.** Le programme est un fichier inerte sur disque, le processus est un programme dynamique en cours d'exécution. · **2.** À sauvegarder et restaurer le contexte lors des commutations. · **3.** La durée de temps processeur allouée à un processus par l'ordonnanceur. · **4.** Les threads partagent la mémoire du parent, les processus lourds sont isolés. · **5.** Une situation d'attente circulaire où des processus se bloquent mutuellement.

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.